SOFTWARE REQUIREMENTS SPECIFICATION

Vehicle Insurance Management System (VIMS)

A Full-Stack Digital Insurance Platform

Submitted By:
Gudipalli Srineel Reddy
March 2026

Table of Contents

1. Introduction....1
1.1. Purpose....1
1.2. Scope....1
1.3. Document Conventions....1
1.4. Intended Audience....1
1.5. References....1
2. Overall Description....1
2.1. Product Perspective....1
2.2. Product Functions....1
2.3. User Classes and Characteristics....1
2.4. Operating Environment....1
2.5. Design and Implementation Constraints....1
3. System Features Functional Requirements....1
3.1. Authentication Module (FR-AUTH)....1
3.2. Policy Plan Management (FR-PLAN)....1
3.3. Vehicle Application (FR-APP)....1
3.4. Policy Management (FR-POL)....1
3.5. Policy Transfer (FR-TRANS)....1
3.6. Claims Management (FR-CLM)....1
3.7. Payment and Invoicing (FR-PAY)....1
3.8. Audit Logging (FR-AUDIT)....1
3.9. Administrator Functions (FR-ADMIN)....1
4. UML Diagrams....1
4.1. Use Case Diagram....1
4.2. Class Diagram....1
4.3. Entity-Relationship (ER) Diagram....1
4.4. Activity Diagram End-to-End Business Process....1
4.5. Sequence Diagram Policy Purchase and Claim Process....1
5. External Interface Requirements....1
5.1. User Interfaces....1
5.2. API Interface....1

5.3. Software Interfaces 1
6. Non-Functional Requirements 1
6.1. Performance 1
6.2. Security 1
6.3. Reliability and Availability 1
6.4. Maintainability and Scalability 1
7. Database Design 1
7.1. Entity Summary 1
7.2. Key Relationships and Cascade Rules 1
8. Use Case Specifications 1
UC-01: Customer Purchases a Policy 1
UC-02: Customer Files an Insurance Claim 1
UC-03: Agent Reviews Vehicle Application 1
9. System Constraints 1
Appendix A: Business Rules 1
Appendix B: Status Enumerations 1
B.1. PolicyStatus 1
B.2. ClaimStatus 1
B.3. ApplicationStatus / PaymentStatus / TransferStatus 1
Appendix C: Audited Actions 1

1. Introduction

1.1. Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for the Vehicle Insurance Management System (VIMS). It serves as the authoritative reference for developers, testers, project managers, and all project stakeholders throughout the design, development, testing, and maintenance lifecycle.

VIMS is a full-stack web application engineered to fully digitize the vehicle insurance lifecycle covering policy discovery, application, premium payment, claim submission, adjudication, and settlement. The SRS captures every user-facing feature and every system-level constraint required to deliver a production-ready insurance platform.

1.2. Scope

The Vehicle Insurance Management System (VIMS) provides a centralized, multi-role digital platform for managing vehicle insurance operations. The system replaces traditional manual, paper-based workflows with automated, data-driven processes.

The scope of VIMS includes:

- **User account management** with four distinct roles: Customer, Agent/Underwriter, Claims Officer, and Administrator.
- **Vehicle application and document upload** with agent-based underwriting approval workflows.
- **Policy lifecycle management,** including creation, activation, annual payment, multi-year support, renewal, cancellation, and peer-to-peer transfer.
- **Automated premium calculation** using a server-enforced Insured Declared Value (IDV) depreciation engine and plan-specific rate formulas.
- **Claims management,** including multi-type claim submission, evidence upload, officer assignment, payout calculation, and adjudication.
- **Payment processing** with PDF invoice generation for every completed transaction.
- **Immutable audit logging** of every critical system action for compliance and fraud prevention.

Out of scope for version 1.0: Real-money payment gateway integration, VAHAN government API integration, mobile native applications, AI/ML fraud detection, and multi-language (i18n) support.

1.3. Document Conventions

Term	Meaning
FR-XXX	Functional Requirement identifier (e.g. FR-AUTH-01)
NFR-XXX	Non-Functional Requirement identifier

Term	Meaning
UC-XX	Use Case identifier
BR-XX	Business Rule identifier
SHALL	Mandatory requirement must be implemented
SHOULD	Recommended implement where feasible
MAY	Optional implement at team discretion
SPA	Single Page Application (Angular frontend)
JWT	JSON Web Token used for stateless authentication
IDV	Insured Declared Value current market value of insured vehicle
RBAC	Role-Based Access Control
EF Core	Entity Framework Core ORM used for all database access

1.4. Intended Audience

Audience	Usage of this Document
Backend Developers	API design, service layer logic, database schema, business rule enforcement
Frontend Developers	Angular component requirements, route guards, HTTP interceptor behaviour
QA / Test Engineers	Test case derivation from FRs, use cases, and business rules
Project Manager	Scope boundary, milestone planning, change control baseline
System Architects	Non-functional requirements, integration points, deployment constraints
Stakeholders / Product Owners	Feature scope, acceptance criteria, expected system behaviour

1.5. References

- ASP.NET Core 8.0 Documentation https://learn.microsoft.com/en-us/aspnet/core
- Angular 20+ Official Docs https://angular.dev
- Entity Framework Core Docs https://learn.microsoft.com/en-us/ef/core
- JWT RFC 7519 https://datatracker.ietf.org/doc/html/rfc7519
- OWASP Top 10 Web Application Security Risks (2021)
- India Motor Tariff IRDA Third-Party Premium Rate Tables (reference)

2. Overall Description

2.1. Product Perspective

VIMS is a standalone web application consisting of two tightly coupled subsystems:

- **Backend REST API:** ASP.NET Core 8.0, structured in Clean Architecture (Service-Repository pattern). Exposes 39 JSON endpoints across five controller groups. Handles all business logic, security enforcement, file storage, and PDF generation.
- **Frontend SPA:** Angular 20+ Single Page Application. Provides role-specific dashboards for all four user roles. Consumes the REST API exclusively. Enforces client-side route guards and JWT injection via HTTP Interceptor.

In a production deployment, the Angular SPA is served as compiled static assets by the same ASP.NET Core host or a CDN. The API is reachable at https://<host>/api. Microsoft SQL Server is hosted on a dedicated database server or cloud managed instance.

2.2. Product Functions

Function Group	Capability Summary
Authentication & Security	Registration, login with CAPTCHA, JWT issuance, password change, security question reset
Policy Plan Catalogue	Admin creates and manages PolicyPlans; Customers browse plans and get instant quotes
Vehicle Application	Customer submits vehicle + document upload; Agent reviews and approves/rejects
Policy Management	Policy creation on approval, annual payments, multi-year tracking, renewal, cancellation
Policy Transfer	Customer-to-customer ownership transfer with dual-consent workflow
Claims Management	Multi-type claim filing, evidence upload, officer assignment, payout adjudication
Payment & Invoicing	Payment record creation, PDF invoice generation and download
Audit Logging	Immutable write-once AuditLog for all critical actions across all roles
Admin Oversight	Full system-wide visibility and control users, plans, policies, claims, payments

2.3. User Classes and Characteristics

Role	Enum Code	Key Characteristics
Administrator	0 (Admin)	Full system control. Creates agents and officers. Views all records. Non-daily operational user.

Role	Enum Code	Key Characteristics
Customer	1 (Customer)	Primary end-user. Purchases policies, files claims, makes payments. Variable technical literacy.
Agent / Underwriter	2 (Agent)	Reviews vehicle applications and documents. Approves or rejects. Professional insurance user.
Claims Officer	3 (ClaimsOfficer)	Reviews and adjudicates claims. Sets approved payout amounts. Professional insurance user.

2.4. Operating Environment

- **Runtime:** ASP.NET Core 8.0 on .NET 8; Windows Server 2022 or Ubuntu 22.04 LTS.
- **Database:** Microsoft SQL Server 2019+ or Azure SQL Database. EF Core Code-First migrations manage schema.
- **Frontend:** Angular 20+ SPA compiled to static HTML/JS/CSS. Requires Node 22+ at build time only.
- **Browser Support:** Chrome 120+, Firefox 120+, Edge 120+, Safari 17+. Internet Explorer is not supported.
- **File Storage:** Local filesystem (wwwroot/uploads/) in . Cloud blob storage in future versions.
- **PDF Generation:** Server-side .NET library (QuestPDF / iText). No client-side PDF rendering required.

2.5. Design and Implementation Constraints

- **All business rules and price calculations** MUST be enforced server-side by the Service layer. Client-submitted financial values are always discarded.
- **AuditLog entries** are immutable. No UPDATE or DELETE operations are permitted on AuditLogs at any layer.
- **EF Core Code-First migrations** must manage all schema changes. Direct SQL DDL on production is prohibited.
- **JWT tokens** must be signed with a strong secret key (minimum 256-bit). Expiry must be configurable.
- **Password storage:** Only BCrypt-hashed values (work factor >= 12) may be stored. Plaintext passwords must never be logged or returned in any API response.
- **CAPTCHA validation:** Must be completed on every login and registration request before any credential processing begins.

3. System Features Functional Requirements

3.1. Authentication Module (FR-AUTH)

ID	Requirement	Priority
FR-AUTH-01	The system SHALL provide a Customer self-registration endpoint accepting FullName, Email, Password, and SecurityQuestion/Answer. AuthService SHALL validate CAPTCHA before creating the account.	High
FR-AUTH-02	The system SHALL provide a login endpoint accepting Email, Password, and CAPTCHA token. On success, it SHALL return a signed JWT containing UserId, Email, and Role claims with a configurable expiry.	High
FR-AUTH-03	The system SHALL provide an Admin-only endpoint for creating Agent and Claims Officer accounts with role assignment.	High
FR-AUTH-04	Authenticated users SHALL be able to change their own password by providing current password plus new password (min 8 characters, at least one digit and one special character).	High
FR-AUTH-05	The system SHALL support a password reset flow via security question. A public endpoint returns the question text for a given email; a second endpoint validates the answer and sets the new password.	High
FR-AUTH-06	Authenticated users SHALL be able to set or update their security question and hashed answer at any time.	Medium
FR-AUTH-07	All protected endpoints SHALL validate the JWT Bearer token via ASP.NET Core middleware. Requests without a valid token SHALL return HTTP 401.	High
FR-AUTH-08	The Angular SPA SHALL store the JWT in localStorage, inject it via HTTP Interceptor on every request, and on 401 responses clear the token and redirect to login.	High
FR-AUTH-09	Angular Route Guards SHALL decode the JWT Role claim and block navigation to routes not permitted for the authenticated role, redirecting to an Unauthorized page.	High

3.2. Policy Plan Management (FR-PLAN)

ID	Requirement	Priority
FR-PLAN-01	Admins SHALL be able to create PolicyPlans with: PlanName, PolicyType (ThirdParty/Comprehensive/ZeroDepreciation), BasePremium, MaxCoverageAmount, DeductibleAmount, coverage flags (OwnDamageCover, TheftCover, ThirdPartyCover), add-on flags (RoadsideAssistance, PersonalAccidentCover), and PolicyDurationMonths.	High

ID	Requirement	Priority
FR-PLAN-02	Admins SHALL be able to activate and deactivate PolicyPlans. Deactivated plans SHALL NOT appear in the public catalogue.	High
FR-PLAN-03	All active PolicyPlans SHALL be browsable by Customers, including unauthenticated visitors, with full plan details.	High
FR-PLAN-04	The system SHALL provide a quote calculation endpoint. PricingService calculates and returns InvoiceAmount from vehicle details + PlanId. No database records are created.	High
FR-PLAN-05	PricingService SHALL calculate: IDV = ExShowroomPrice x (1 - DepreciationRate), OD Premium = IDV x OD Rate, TP Premium = Govt-regulated fixed rate by cubic capacity, Final = OD + TP + Add-ons - Discounts. Client price values are ignored.	High
FR-PLAN-06	Depreciation rates SHALL follow IRDA grid: 0-6m=5%, 6m-1yr=15%, 1-2yr=20%, 2-3yr=30%, 3-4yr=40%, 4-5yr=50%, >5yr=60%.	High

3.3. Vehicle Application (FR-APP)

ID	Requirement	Priority
FR-APP-01	Authenticated Customers SHALL be able to submit a VehicleApplication with: RegistrationNumber, Make, Model, Year, FuelType, VehicleType, KilometersDriven, ExShowroomPrice, PlanId, SelectedYears, and at least one uploaded VehicleDocument.	High
FR-APP-02	The system SHALL accept multi-file document uploads (RC Book, ID Proof, Photographs) stored under wwwroot/uploads/{customerId}/vehicle/{timestamp}_{filename}. The DB stores relative paths only.	High
FR-APP-03	On submission, Status = Pending. An AgentId SHALL be assigned (auto or Admin-manual).	High
FR-APP-04	Agents SHALL see all VehicleApplications assigned to them, filterable by status.	High
FR-APP-05	Agents SHALL be able to access all VehicleDocuments linked to an application for review.	High
FR-APP-06	Agents SHALL be able to Approve an application, triggering automatic creation of Vehicle and Policy records. PricingService sets all financial fields on the Policy.	High
FR-APP-07	Agents SHALL be able to Reject an application with a mandatory RejectionReason (minimum 10 characters). Customers SHALL see the rejection reason.	High

3.4. Policy Management (FR-POL)

ID	Requirement	Priority
FR-POL-01	On agent approval, the system SHALL automatically create a Policy with: PolicyNumber (unique, VMS-YYYY-XXXXXXXX format), Status=PendingPayment, PremiumAmount, IDV, InvoiceAmount, StartDate, EndDate, CurrentYearNumber=1, CurrentYearEndDate, IsCurrentYearPaid=false, ClaimCount=0.	High
FR-POL-02	Customers SHALL be able to pay the annual premium. On payment, IsCurrentYearPaid=true, a Payment record is created, and Status transitions to Active.	High
FR-POL-03	When CurrentYearEndDate is reached, Status SHALL transition to PendingRenewal and IsCurrentYearPaid to false.	High
FR-POL-04	Customers SHALL be able to renew a PendingRenewal policy. The system SHALL increment CurrentYearNumber, set new CurrentYearEndDate, and create a renewal Payment record.	Medium
FR-POL-05	Customers SHALL be able to cancel an Active policy. Status transitions to Cancelled.	Medium
FR-POL-06	When EndDate is reached, Status SHALL transition to Expired.	High
FR-POL-07	Customers SHALL be able to view all their own policies with full details, grouped by status.	High
FR-POL-08	Admins SHALL be able to view all policies across all customers with filtering by status and date range.	Medium

3.5. Policy Transfer (FR-TRANS)

ID	Requirement	Priority
FR-TRANS-01	Customers SHALL be able to initiate a policy transfer by specifying a RecipientEmail. The system SHALL validate that the target user exists, is active, and has the Customer role.	Medium
FR-TRANS-02	On initiation, a PolicyTransfer record is created with Status=Pending. The recipient SHALL see the incoming transfer in their dashboard.	Medium
FR-TRANS-03	The recipient SHALL be able to Accept or Reject the transfer. On Accept, the policy CustomerId is updated to the recipient and Status becomes Completed.	Medium
FR-TRANS-04	On Reject, PolicyTransfer Status becomes Rejected with RejectionReason. The original customer retains ownership.	Medium
FR-TRANS-	Only policies with Status=Active are eligible for transfer. Expired,	Medium

ID	Requirement	Priority
05	Cancelled, and PendingPayment policies SHALL NOT be transferable.	

3.6. Claims Management (FR-CLM)

ID	Requirement	Priority
FR-CLM-01	Customers SHALL be able to submit a claim against an Active policy with: ClaimType (OwnDamage/Theft/ThirdParty), IncidentDate, IncidentDescription, ClaimedAmount, and at least one ClaimDocument upload.	High
FR-CLM-02	ClaimsService SHALL validate ClaimType against the PolicyPlan's coverage flags. Claims for uncovered types SHALL return HTTP 422.	High
FR-CLM-03	ClaimsService SHALL prevent duplicate claims: if a Pending or UnderReview claim exists for the same PolicyId + ClaimType, the new submission SHALL return HTTP 409.	High
FR-CLM-04	On successful submission, a ClaimNumber (CLM-YYYY-XXXXXXXX) is assigned, Status=Pending, and ClaimCount on the Policy increments by 1.	High
FR-CLM-05	Claims Officers SHALL see all claims system-wide and can self-assign claims (Status transitions to UnderReview, ClaimsOfficerId set).	High
FR-CLM-06	Officers SHALL Approve a claim by setting ApprovedAmount. ClaimsService SHALL enforce: ApprovedAmount <= MaxCoverageAmount - DeductibleAmount. Exceeding this cap returns HTTP 422.	High
FR-CLM-07	On approval, the system SHALL create a Payment record for ApprovedAmount. Claim Status transitions to Approved then Closed.	High
FR-CLM-08	Officers SHALL Reject a claim with a mandatory DecisionType and RejectionReason. Status transitions to Rejected.	High
FR-CLM-09	Customers SHALL view all their own claims with real-time status. Officers SHALL see their caseload via a My Claims view.	High
FR-CLM-10	Claim documents SHALL be stored under wwwroot/uploads/{customerId}/claims/{claimId}/{timestamp}_{filename}.	High

3.7. Payment and Invoicing (FR-PAY)

ID	Requirement	Priority
FR-PAY-01	Every premium payment and claim settlement SHALL create a Payment record with: PolicyId, Amount, PaymentDate, PaymentMethod, Status=Completed, TransactionReference (UUID, unique).	High
FR-PAY-02	Customers SHALL be able to view their complete payment history.	High
FR-PAY-03	Customers SHALL be able to download a PDF invoice for any completed payment. The invoice SHALL display: PolicyNumber, Customer name, vehicle details, PaymentDate, Amount, PaymentMethod, and TransactionReference.	High

ID	Requirement	Priority
FR-PAY-04	The PDF SHALL be generated server-side and returned as application/pdf. No pre-generation; invoices are on-demand.	Medium
FR-PAY-05	Admins SHALL be able to view all Payment records system-wide, filterable by date range and PaymentMethod.	Medium

3.8. Audit Logging (FR-AUDIT)

ID	Requirement	Priority
FR-AUDIT-01	The system SHALL write an AuditLog entry for every critical action in Appendix C. Each entry SHALL contain: UserId (nullable), Email, Role, Action, Category, EntityName, EntityId, Details (JSON), Timestamp (UTC).	High
FR-AUDIT-02	AuditLog entries SHALL be immutable. No UPDATE or DELETE operations shall exist at any application layer.	High
FR-AUDIT-03	Admins SHALL be able to query AuditLogs with filters: date range, Category, UserId, EntityName, EntityId.	Medium
FR-AUDIT-04	AuditLog writes SHALL NOT block the primary operation. Write failures SHALL be logged to a secondary error log but SHALL NOT roll back the parent transaction.	High

3.9. Administrator Functions (FR-ADMIN)

ID	Requirement	Priority
FR-ADMIN-01	Admins SHALL be able to view all registered users with their roles and active/inactive status.	High
FR-ADMIN-02	Admins SHALL be able to activate and deactivate any user. Deactivated users SHALL receive HTTP 401 on the next API call.	High
FR-ADMIN-03	Admins SHALL be able to view all VehicleApplications, Policies, Claims, and Payments system-wide.	High
FR-ADMIN-04	Admins SHALL be able to view full AuditLog history with advanced filtering.	Medium
FR-ADMIN-05	Admin endpoints SHALL only be accessible to Role=Admin (role=0). Non-Admin requests SHALL return HTTP 403.	High

4. UML Diagrams

The diagrams in this section provide a comprehensive visual model of VIMS from multiple perspectives actor interactions, domain model, entity relationships, operational workflow, and time-ordered message sequences. Together they form the complete system model that complements the textual requirements in Section 3.

4.1. Use Case Diagram

The Use Case Diagram captures all interactions between the four system actors (Admin, Agent, Customer, Claims Officer) and the system's core functional use cases. It provides a quick visual overview of which role can perform which operations. Shared use cases such as Login illustrate common authentication entry points across multiple roles.

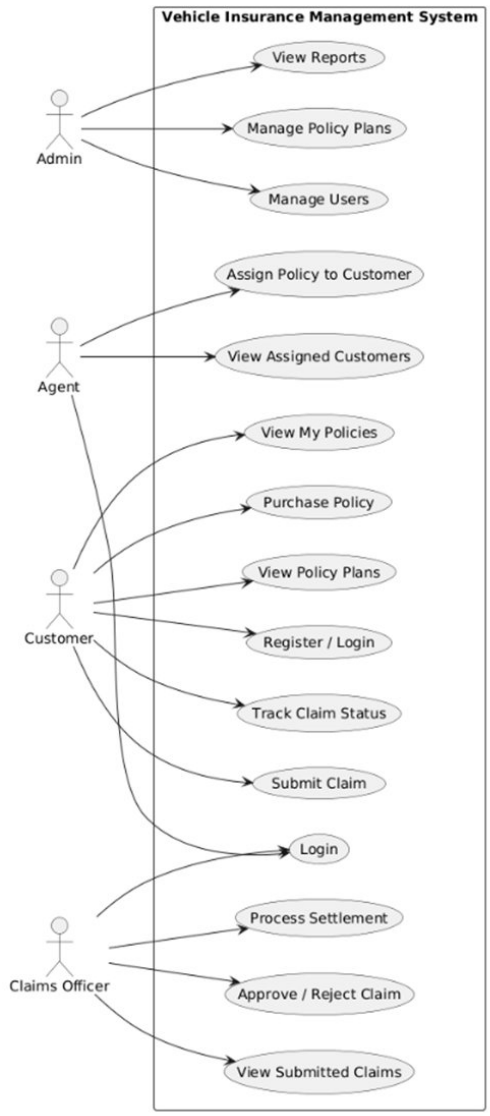

Figure 1: Use Case Diagram Vehicle Insurance Management System

4.2. Class Diagram

The Class Diagram illustrates the core domain model the primary entity classes, their attributes with full type signatures, and the navigable associations between them. Cardinality labels (1, 0..*) and role names (owns, insured under, defines, generates, has, contains, files, handles) describe how entities relate and navigate to each other within the application's service layer.

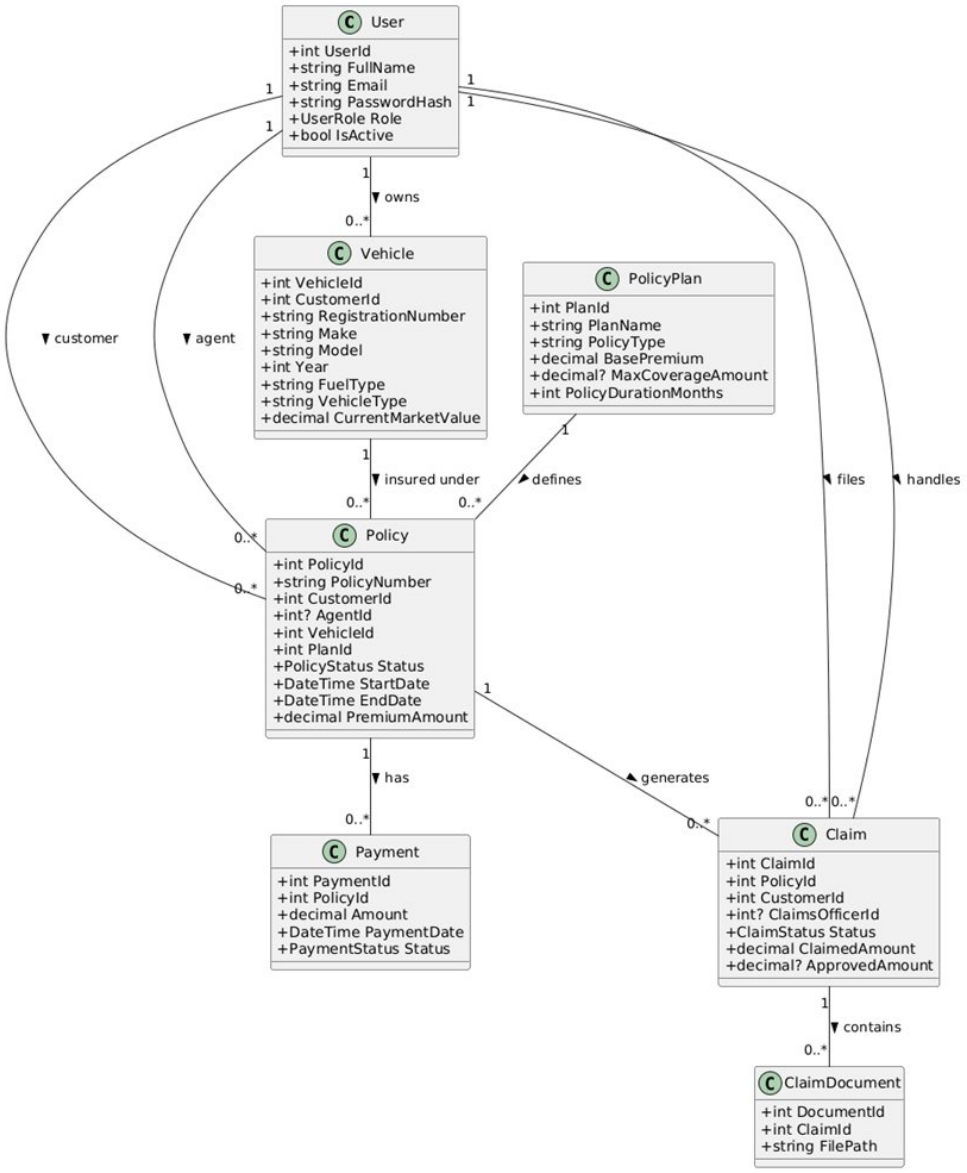

Figure 2: Class Diagram VIMS Core Domain Model

4.3. Entity-Relationship (ER) Diagram

The ER Diagram provides the formal data-model view of the VehicleInsuranceDB schema as implemented in Microsoft SQL Server via EF Core Code-First migrations. Each entity is shown with its primary key (PK) and foreign key (FK) attributes, data types, and crow's foot notation cardinality. The schema is normalized to Third Normal Form (3NF) to eliminate redundancy and enforce referential integrity.

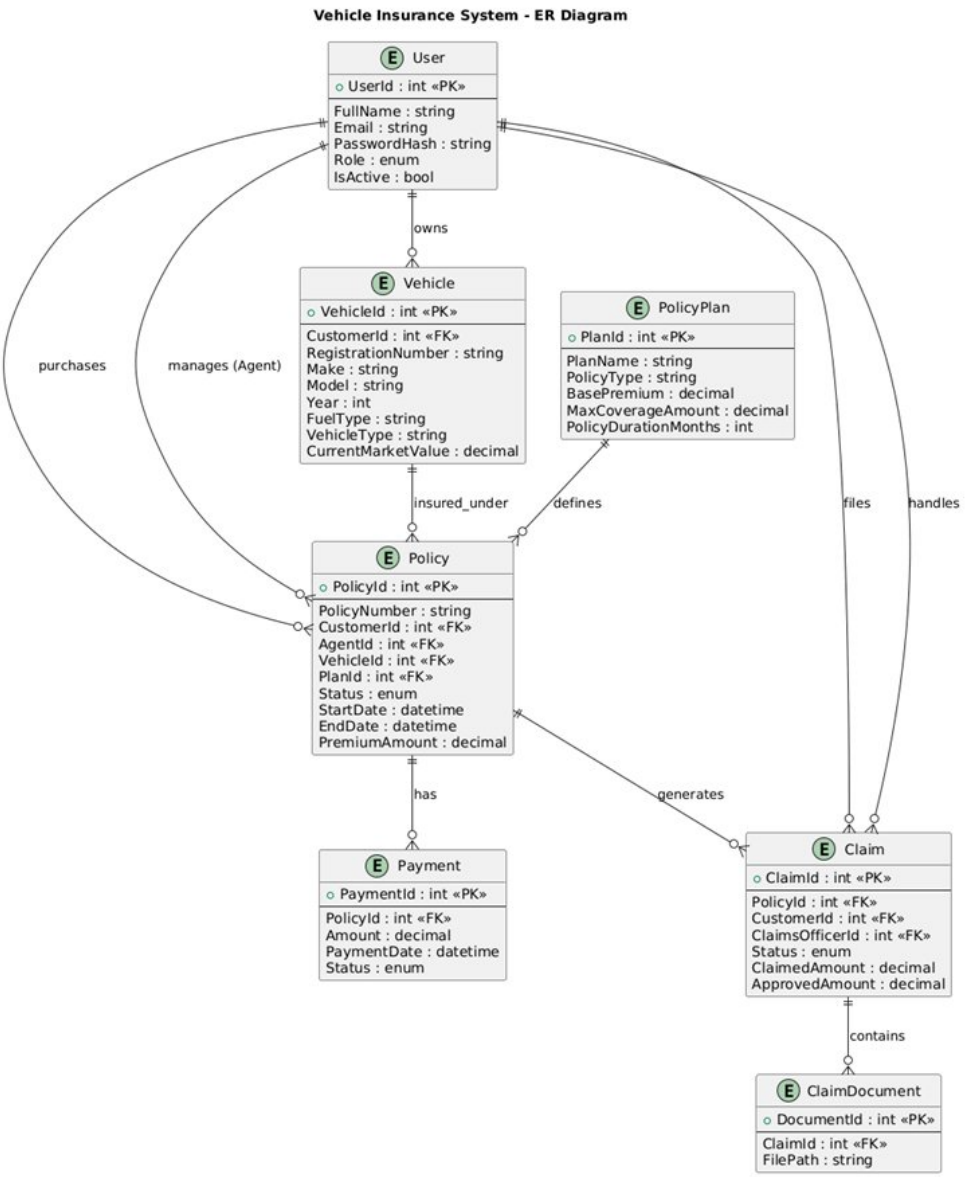

Figure 3: Entity-Relationship Diagram VehicleInsuranceDB Schema

4.4. Activity Diagram End-to-End Business Process

The Activity Diagram traces the complete end-to-end operational flow of VIMS: from user registration through policy purchase, claim submission, claims officer adjudication, and final policy lifecycle termination. Decision nodes (Role?, Payment Successful?, Documents Valid?, Policy Duration Over?) show the conditional branching that drives state transitions across the entire system. Both Customer/Agent and Claims Officer parallel paths are shown.

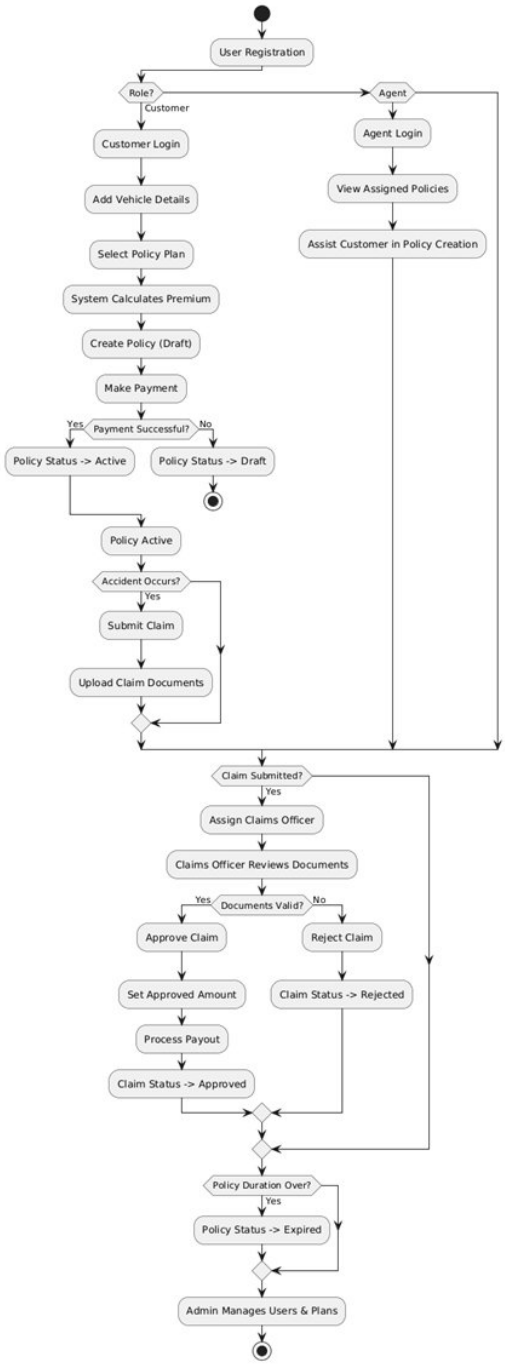

Figure 4: Activity Diagram VIMS End-to-End Process Flow

4.5. Sequence Diagram Policy Purchase and Claim Process

The Sequence Diagram shows the time-ordered message flow between four participants Customer, Insurance System (ASP.NET Core API), Payment Gateway, and Claims Officer across two interaction scenarios. The Policy Purchase scenario illustrates the complete registration through policy activation flow. The Claim Process scenario illustrates submission, officer assignment, review, and approve/reject adjudication with UML alt fragments for each decision branch.

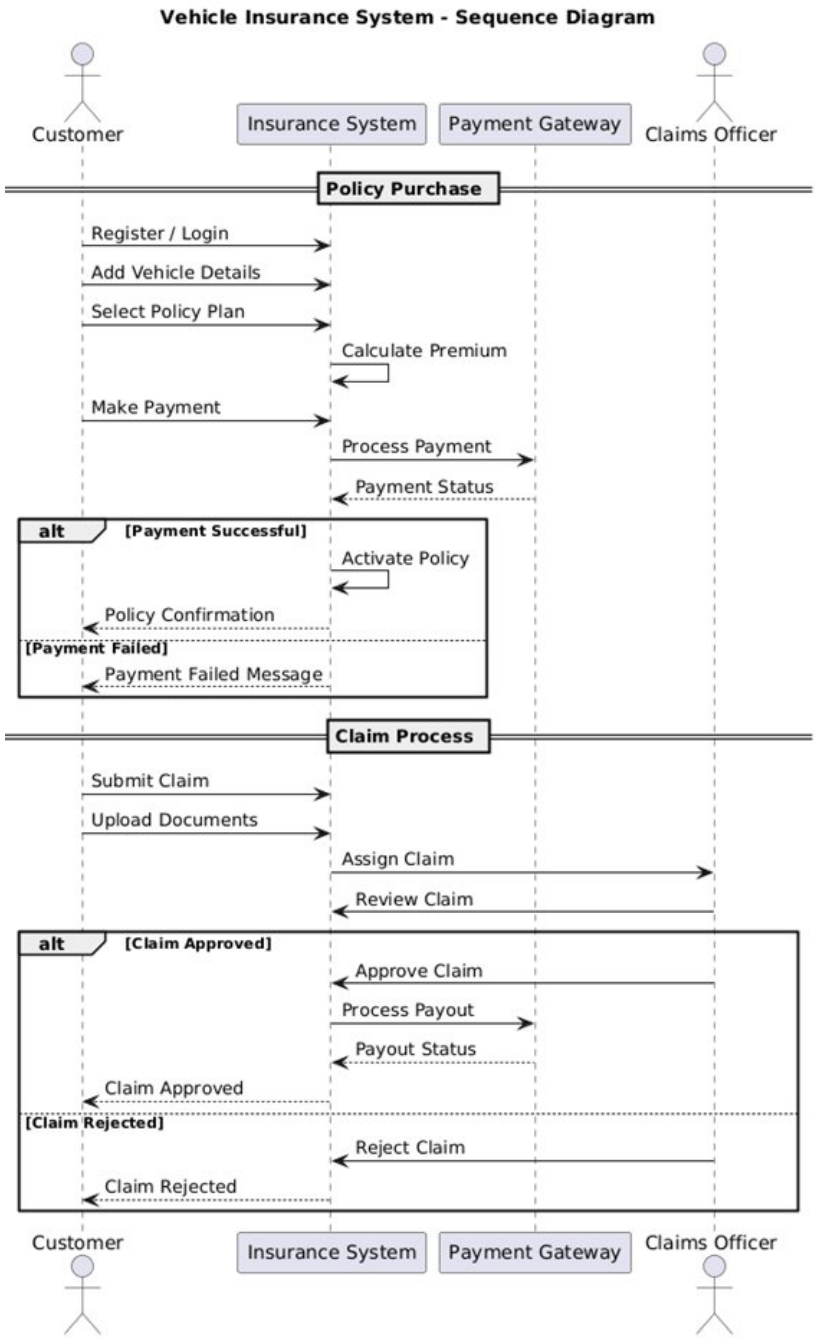

Figure 5: Sequence Diagram Policy Purchase and Claim Process

5. External Interface Requirements

5.1. User Interfaces

Interface	Description
Customer Dashboard	Navigation: My Applications, My Policies, My Claims, Payment History, Explore Plans. Responsive layout with Tailwind CSS.
Agent Dashboard	Navigation: Pending Applications, All Applications. Document viewer. Approve/Reject action panel with rejection reason field.
Claims Officer Dashboard	Navigation: All Claims Queue, My Claims. Full claim detail with payout entry and decision panel.
Admin Dashboard	Navigation: Manage Users, Manage Plans, All Policies, All Claims, All Payments, Audit Logs. Paginated data tables with filtering.
Explore Plans Page	Public (no auth required). Plan cards with full details. Quote Calculator widget using live PricingService.
Error Pages	401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error styled Angular route components.

5.2. API Interface

All API communication uses JSON over HTTPS. The API base URL is https://<host>/api. Authentication uses the Authorization: Bearer <JWT> header. All timestamps are ISO 8601 UTC. File uploads use multipart/form-data. Swagger UI is available at /swagger/index.html in development.

Error response schema: { "statusCode": 422, "message": "Claim type not covered by plan.", "errors": {} }

5.3. Software Interfaces

External System	Interface Type	Purpose in VIMS
Microsoft SQL Server	EF Core / ADO.NET	Primary relational data store for all entities
CAPTCHA Provider (reCAPTCHA v3)	HTTPS REST	Token validation on login and registration
PDF Library (QuestPDF / iText)	In-process .NET	Server-side PDF invoice generation
Email / SMTP	Future scope	Notification emails for approvals, rejections, claim decisions
Payment Gateway	Future scope	Real-money premium and claim settlement processing
VAHAN Government	Future scope	Automatic vehicle registration detail lookup

External System	Interface Type	Purpose in VIMS
API		

6. Non-Functional Requirements

6.1. Performance

ID	Requirement	Target
NFR-PERF-01	API response time for standard CRUD endpoints	< 300 ms at p95 under 100 concurrent users
NFR-PERF-02	Premium calculation quote endpoint	< 500 ms including IDV computation
NFR-PERF-03	PDF invoice generation	< 3 seconds for a standard invoice
NFR-PERF-04	File upload endpoint (single document, up to 10 MB)	< 5 seconds on standard broadband
NFR-PERF-05	Angular SPA first contentful paint	< 3 seconds on a 10 Mbps connection
NFR-PERF-06	Indexed database lookups (PolicyNumber, Email, ClaimNumber, TransactionReference)	< 100 ms per query

6.2. Security

ID	Requirement
NFR-SEC-01	All endpoints SHALL use HTTPS. HTTP connections SHALL redirect to HTTPS (HSTS enforced).
NFR-SEC-02	Passwords SHALL be hashed with BCrypt (work factor >= 12). Plaintext passwords SHALL never be stored, logged, or returned.
NFR-SEC-03	JWT tokens SHALL be signed with HMAC-SHA256 (minimum 256-bit key), expiry <= 24 hours. Validation SHALL check signature, expiry, and IsActive.
NFR-SEC-04	All database queries SHALL use parameterized queries via EF Core. String interpolation in SQL is prohibited.
NFR-SEC-05	File uploads SHALL be restricted to: .pdf, .jpg, .jpeg, .png only. Maximum file size = 10 MB per file.
NFR-SEC-06	CORS policy SHALL restrict allowed origins to the production Angular domain.
NFR-SEC-07	CAPTCHA validation SHALL protect all login and registration endpoints.
NFR-SEC-08	Financial values submitted by clients (InvoiceAmount, PremiumAmount, ApprovedAmount) SHALL be ignored; server-computed values always override.

6.3. Reliability and Availability

ID	Requirement
NFR-REL-01	System SHALL target 99.5% uptime in production (planned maintenance excluded).
NFR-REL-02	Retry logic (3 retries, exponential backoff) SHALL be implemented for transient SQL Server connectivity failures.
NFR-REL-03	Critical operations (policy creation, payment, claim updates) SHALL be wrapped in database transactions. Partial state SHALL never persist.
NFR-REL-04	AuditLog write failures SHALL NOT cause rollback of the parent business transaction (non-blocking audit pattern).

6.4. Maintainability and Scalability

ID	Requirement
NFR-MAINT-01	All business logic SHALL reside in the Service layer. Zero business logic in Controllers or Repositories.
NFR-MAINT-02	All database access SHALL go through Repository interfaces. No raw DbContext calls in Controller or Service classes.
NFR-MAINT-03	The system SHOULD support horizontal scaling without shared in-memory state. JWT is stateless. File storage SHALL support shared storage in multi-instance deployments.
NFR-MAINT-04	EF Core Code-First migrations SHALL be the only mechanism for schema changes.
NFR-MAINT-05	Minimum 75% unit test coverage for all Service layer methods (PricingService, ClaimsService, AuthService).

7. Database Design

The VehicleInsuranceDB database is managed by EF Core Code-First migrations on Microsoft SQL Server. The schema is normalized to Third Normal Form (3NF). Financial fields use DECIMAL(18,2) to prevent floating-point rounding errors.

7.1. Entity Summary

Table	PK	Key Foreign Keys	Description
Users	UserId	None	All users. Role: 0=Admin, 1=Customer, 2=Agent, 3=ClaimsOfficer. Email UNIQUE. IsActive.
PolicyPlans	PlanId	None	Plan templates. Coverage flags: OwnDamageCover, TheftCover, ThirdPartyCover. Add-ons: RoadsideAssistance, PersonalAccidentCover.
VehicleApplications	ApplicationId	CustomerId, AssignedAgentId, PlanId	Customer applications. Status: Pending/Approved/Rejected. InvoiceAmount set by PricingService.
VehicleDocuments	DocumentId	VehicleApplicationId (CASCADE)	Uploaded docs for an application. DocumentType, FilePath (relative).
Vehicles	VehicleId	CustomerId, VehicleApplicationId (UNIQUE)	Registered vehicles. RegistrationNumber UNIQUE. CurrentMarketValue = IDV at approval time.
Policies	PolicyId	CustomerId, AgentId, VehicleId (CASCADE), PlanId (CASCADE)	Active contracts. PolicyNumber UNIQUE. Multi-year: SelectedYears, CurrentYearNumber, CurrentYearEndDate, IsCurrentYearPaid.
Claims	ClaimId	PolicyId (CASCADE), CustomerId, ClaimsOfficerId (nullable)	Claim records. ClaimNumber UNIQUE. ApprovedAmount enforced <= MaxCoverage - Deductible.
Payments	PaymentId	PolicyId (CASCADE)	All transactions. TransactionReference UNIQUE (UUID). PaymentMethod: Premium / ClaimSettlement.
PolicyTransfers	TransferId	PolicyId,	Ownership transfer requests.

Table	PK	Key Foreign Keys	Description
		SenderCustomerId, RecipientCustomerId	Status: Pending/Completed/Rejected.
AuditLogs	AuditId	UserId (SetNull)	Immutable write-once log. Never deleted or updated. Timestamp UTC.

7.2. Key Relationships and Cascade Rules

Parent	Child	Cardinality	Cascade Rule
Users	VehicleApplications	1:Many	None
VehicleApplications	VehicleDocuments	1:Many	CASCADE DELETE
VehicleApplications	Vehicles	1:1	None Vehicle persists after approval
Vehicles	Policies	1:Many	CASCADE DELETE
PolicyPlans	Policies	1:Many	CASCADE DELETE
Policies	Claims	1:Many	CASCADE DELETE
Policies	Payments	1:Many	CASCADE DELETE
Policies	PolicyTransfers	1:Many	None transfer history retained for audit
Users	AuditLogs	1:Many	SetNull logs retained even if user deleted

8. Use Case Specifications

UC-01: Customer Purchases a Policy

Field	Detail
Primary Actor	Customer
Secondary Actor	Agent
Precondition	Customer is registered and logged in. At least one active PolicyPlan exists.
Trigger	Customer clicks Apply for Insurance from the Explore Plans page.
Main Flow	1. Customer selects a PolicyPlan and opens the vehicle application form. 2. Customer enters vehicle details and uploads at least one document (RC Book, ID Proof). 3. System calls PricingService and displays the calculated InvoiceAmount (read-only). 4. Customer submits the application. Status=Pending. AgentId auto-assigned. 5. Agent reviews application details and all uploaded documents. 6. Agent approves system creates Vehicle + Policy (Status=PendingPayment). 7. Customer pays the InvoiceAmount Policy Status transitions to Active.
Alternate Flow Rejection	Step 6a: Agent rejects. RejectionReason recorded. Status=Rejected. Customer notified.
Postcondition	Policy is Active. Payment record created. AuditLog entries written for approval and payment.
Applicable Business Rules	BR-04 (server-side pricing), BR-06 (min 1 document required), BR-08 (date calculation)

UC-02: Customer Files an Insurance Claim

Field	Detail
Primary Actor	Customer
Secondary Actor	Claims Officer
Precondition	Customer has an Active policy. ClaimType is covered by the linked PolicyPlan.
Trigger	Customer clicks File a Claim on their policy detail page.
Main Flow	1. Customer selects ClaimType, enters IncidentDate, IncidentDescription, ClaimedAmount. 2. Customer uploads at least one ClaimDocument (photo, FIR copy, repair estimate). 3. ClaimsService validates coverage and checks for duplicate active claims.

Field	Detail
	4. Claim created (Status=Pending). ClaimCount on Policy incremented. AuditLog written. 5. Claims Officer self-assigns the claim. Status transitions to UnderReview. 6. Officer reviews all ClaimDocuments and calculates ApprovedAmount. 7. ClaimsService validates ApprovedAmount <= MaxCoverageAmount - DeductibleAmount. 8. Status=Approved. Payment record auto-created for ApprovedAmount. Status=Closed.
Alternate Flows	Step 3a: Coverage not included HTTP 422, claim rejected before creation. Step 8a: Officer rejects RejectionReason recorded, Status=Rejected.
Postcondition	Claim closed. Settlement Payment created. AuditLog entries for all decisions.
Applicable Business Rules	BR-07 (min 1 doc), BR-09 (coverage check), BR-10 (duplicate check), BR-11 (payout cap)

UC-03: Agent Reviews Vehicle Application

Field	Detail
Primary Actor	Agent
Precondition	Agent is logged in. A VehicleApplication with Status=Pending is assigned to this agent.
Trigger	Agent opens the Pending Applications list and selects an application.
Main Flow	1. Agent views application details: customer info, vehicle specs, selected plan. 2. Agent downloads and reviews all uploaded VehicleDocuments. 3. Agent clicks Approve system auto-creates Vehicle and Policy records.
Alternate Flow	Step 3a: Agent clicks Reject enters mandatory RejectionReason (>=10 chars). Status=Rejected.
Postcondition	Application Status updated. If Approved: Vehicle + Policy created. AuditLog written.
Applicable Business Rules	BR-04 (PricingService sets financials), BR-06 (min 1 document required)

9. System Constraints

ID	Constraint	Description
C-01	No Client-Side Pricing	PricingService MUST be the only source of InvoiceAmount, PremiumAmount, and IDV. Client-submitted financial values are always discarded by the API.
C-02	Immutable AuditLog	The AuditLogs table has no UPDATE or DELETE endpoints at any layer. This is a hard architectural constraint enforced by code review.
C-03	Migrations Only	All database schema changes must go through EF Core Code-First migrations. Direct SQL DDL in production is prohibited.
C-04	BCrypt Hashing	Passwords must be stored as BCrypt hashes (work factor >=12). No other hashing algorithm is permitted for passwords.
C-05	File Type Whitelist	Upload endpoints only accept: .pdf, .jpg, .jpeg, .png. Other MIME types return HTTP 415.
C-06	Single Currency	All financial values are in Indian Rupees (INR). Multi-currency is out of scope for .
C-07	Stateless API	The API is stateless. No server-side session state. All auth state lives in the signed JWT.
C-08	SQL Server Only	Database layer targets Microsoft SQL Server. No other RDBMS is supported in .

Appendix A: Business Rules

ID	Business Rule
BR-01	IDV is server-calculated: IDV = ExShowroomPrice x (1 - IRDA Depreciation Rate). Client values are never used.
BR-02	PolicyNumber format: VMS-YYYY-XXXXXXXX (year + 8-digit zero-padded sequence). Must be globally unique.
BR-03	ClaimNumber format: CLM-YYYY-XXXXXXXX (year + 8-digit sequence). Must be globally unique.
BR-04	InvoiceAmount and PremiumAmount are set exclusively by PricingService on the server. Client-submitted financial amounts are ignored at all API endpoints.
BR-05	A policy with ClaimCount > 0 is ineligible for a No-Claim Bonus (NCB) discount on renewal.
BR-06	A VehicleApplication must have at least one VehicleDocument attached. Applications without documents are rejected with HTTP 422.
BR-07	A Claim must have at least one ClaimDocument attached at submission time. Submissions without evidence return HTTP 422.
BR-08	Policy StartDate = the date of agent approval. EndDate = StartDate + (PolicyDurationMonths x SelectedYears). CurrentYearEndDate = StartDate + 12 months.
BR-09	ClaimsService validates ClaimType against PolicyPlan coverage flags: OwnDamage requires OwnDamageCover=true, Theft requires TheftCover=true, ThirdParty requires ThirdPartyCover=true.
BR-10	A claim cannot be filed if a Pending or UnderReview claim of the same ClaimType already exists for the same PolicyId. Returns HTTP 409.
BR-11	ApprovedAmount set by a Claims Officer must satisfy: ApprovedAmount <= MaxCoverageAmount - DeductibleAmount. ClaimsService enforces this cap; exceeding it returns HTTP 422.
BR-12	A PolicyTransfer can only be initiated for policies with Status=Active. PendingPayment, Expired, and Cancelled policies are not transferable.
BR-13	The transfer recipient must be a registered, active user with Role=Customer. Agents, Officers, and Admins cannot receive transferred policies.
BR-14	An Admin can activate or deactivate any user. Deactivated users' JWTs are rejected on the next API call (IsActive checked in JWT validation middleware).
BR-15	Deactivated PolicyPlans do not appear in the public catalogue. Existing policies linked to a deactivated plan remain unaffected.
BR-16	AuditLog entries are write-once and immutable. No application-level update or delete operations exist for AuditLogs. Database cascades that would delete audit records are

ID	Business Rule
	prohibited.

Appendix B: Status Enumerations

B.1. PolicyStatus

Value	Numeric	Meaning	Allowed Transitions
PendingPayment	0	Policy created; awaiting first premium payment	Active (on payment)
Active	1	Policy is live and fully operative	PendingRenewal, Cancelled, Expired, Transferred
PendingRenewal	2	CurrentYearEndDate reached; renewal premium due	Active (on renewal payment)
Cancelled	3	Cancelled by customer	Terminal no further transitions
Expired	4	EndDate reached	Terminal no further transitions
Transferred	5	Ownership transferred to another customer	Terminal original policy record retained

B.2. ClaimStatus

Value	Numeric	Meaning	Allowed Transitions
Pending	0	Filed by customer; awaiting officer assignment	UnderReview
UnderReview	1	Officer assigned and reviewing documents	Approved, Rejected
Approved	2	Officer approved; settlement payment created	Closed
Rejected	3	Officer rejected with documented reason	Terminal
Closed	4	Settlement payment confirmed; claim fully resolved	Terminal

B.3. ApplicationStatus / PaymentStatus / TransferStatus

Enum	Values
ApplicationStatus	Pending | Approved | Rejected
PaymentStatus	Pending | Completed | Failed

Enum	Values
TransferStatus	Pending | Completed | Rejected
PaymentMethod	PremiumPayment | RenewalPayment | ClaimSettlement
ClaimType	OwnDamage | Theft | ThirdParty
DecisionType	FullSettlement | PartialSettlement | Rejection

Appendix C: Audited Actions

The following actions SHALL each generate an immutable AuditLog entry per FR-AUDIT-01. The AuditLogService is called as the final step of each operation before the API response is returned.

Category	Action	Entity	Key Details Captured
Auth	UserRegistered	Users	UserId, Email, Role
Auth	UserLoggedIn	Users	UserId, Email
Auth	PasswordChanged	Users	UserId, Email
Auth	PasswordReset	Users	UserId, Email
Auth	UserActivated / UserDeactivated	Users	AdminUserId, TargetUserId, TargetEmail
Application	ApplicationSubmitted	VehicleApplications	ApplicationId, CustomerId, PlanId, SelectedYears
Application	ApplicationApproved	VehicleApplications	ApplicationId, AgentId, VehicleId, PolicyId
Application	ApplicationRejected	VehicleApplications	ApplicationId, AgentId, RejectionReason
Policy	PolicyCreated	Policies	PolicyId, PolicyNumber, CustomerId, PlanId
Policy	PolicyPaymentMade	Policies	PolicyId, PaymentId, Amount, PaymentMethod
Policy	PolicyRenewed	Policies	PolicyId, CurrentYearNumber, PaymentId
Policy	PolicyCancelled	Policies	PolicyId, CustomerId
Policy	PolicyExpired	Policies	PolicyId, EndDate
Transfer	TransferInitiated	PolicyTransfers	TransferId, PolicyId, SenderCustomerId, RecipientCustomerId
Transfer	TransferAccepted / TransferRejected	PolicyTransfers	TransferId, PolicyId, DecisionUserId
Claim	ClaimSubmitted	Claims	ClaimId, ClaimNumber, PolicyId, ClaimType, ClaimedAmount
Claim	ClaimAssigned	Claims	ClaimId, ClaimsOfficerId
Claim	ClaimApproved	Claims	ClaimId, ClaimsOfficerId, ApprovedAmount, PaymentId

Category	Action	Entity	Key Details Captured
Claim	ClaimRejected	Claims	ClaimId, ClaimsOfficerId, RejectionReason, DecisionType
Plan	PolicyPlanCreated	PolicyPlans	PlanId, PlanName, PolicyType, AdminUserId
Plan	PolicyPlanDeactivated	PolicyPlans	PlanId, AdminUserId

End of Document

VIMS Software Requirements Specification | March 2026